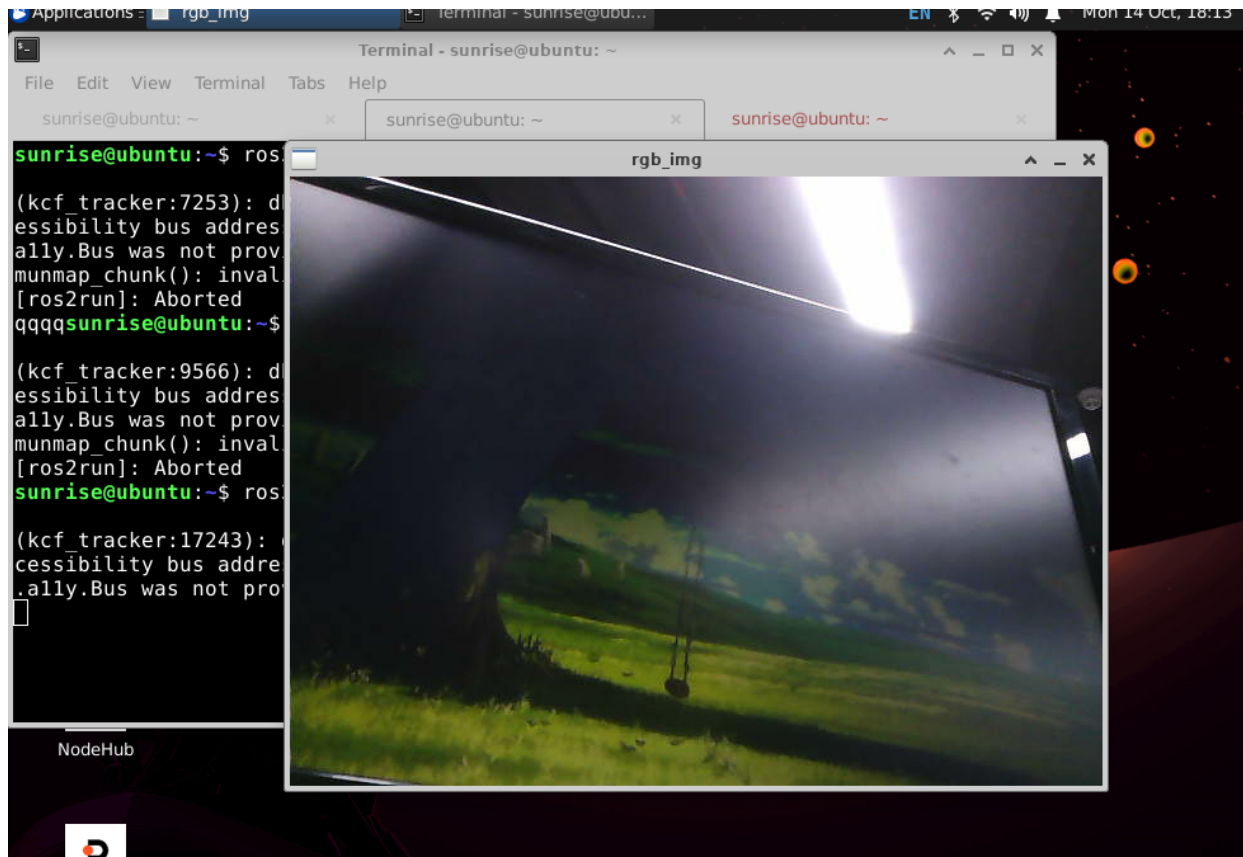


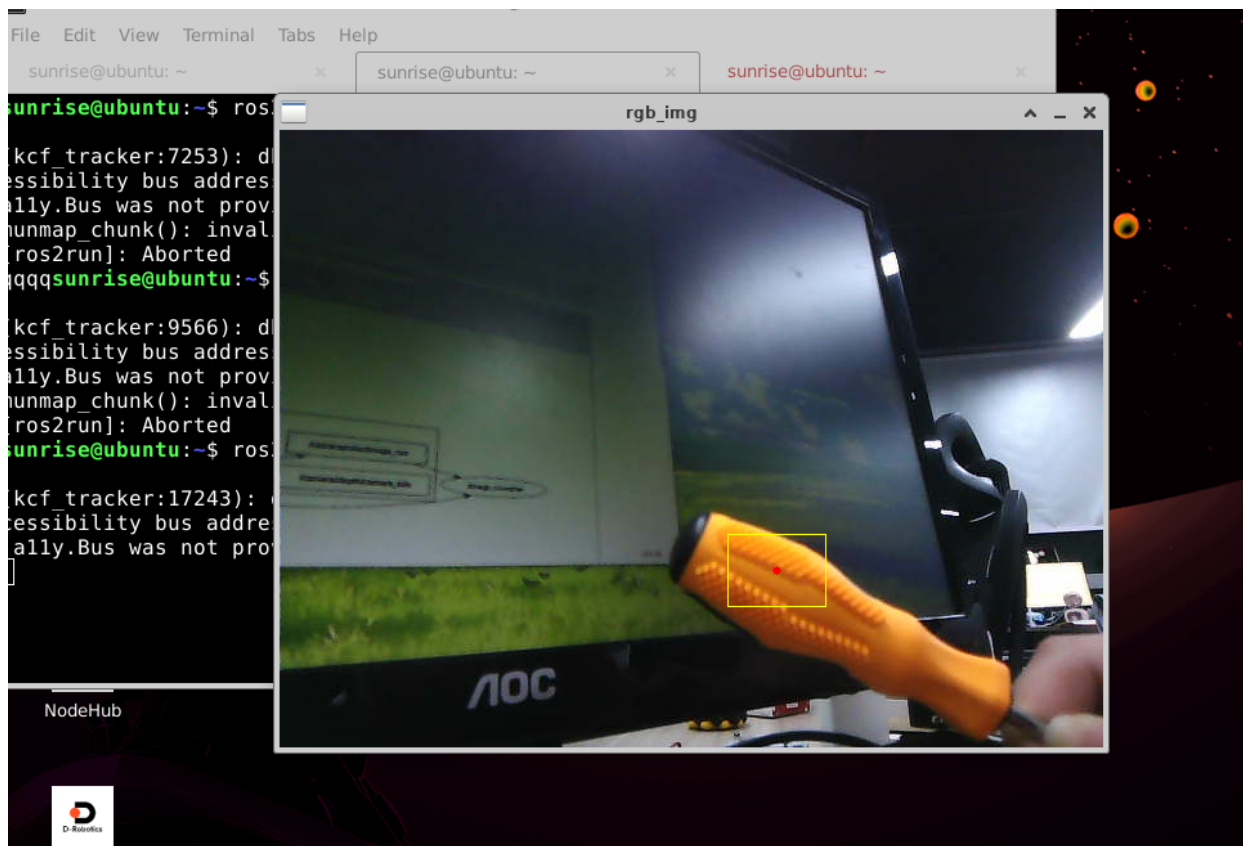
4.KCF object tracking

1. Start program

```
ros2 launch astra_camera gemini.launch.xml  
#Start object tracking node  
ros2 run yahboomcar_kcftracker kcf_tracker
```



After the program is successfully launched, enter the image shown in the figure above. Select the object to be tracked with the mouse, and release it to frame it, as shown in the figure below.

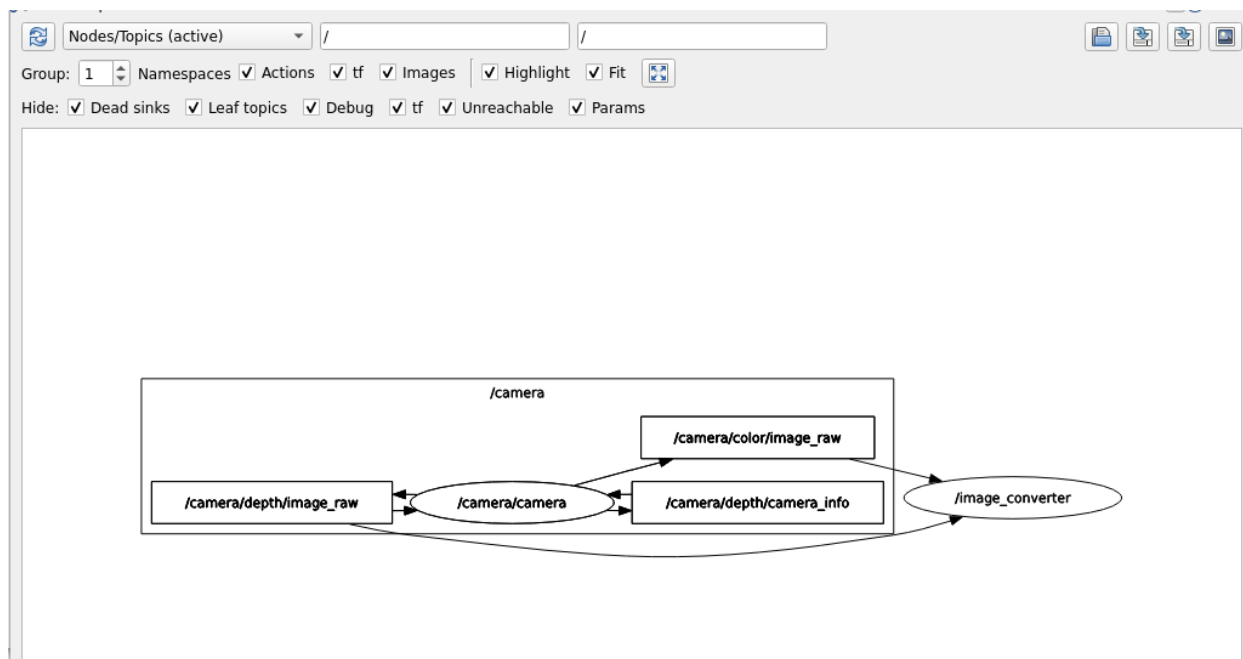


At this point, moving the object box will follow suit, and this feature can be applied to small cars.

View communication between nodes.

Input following command in terminal.

```
ros2 run rqt_graph rqt_graph
```



4.2、 Core code

Code path

```
~/orbbec_ws/src/yahboomcar_kcftracker/src/KCF_Tracker.cpp
```

The principle of functional implementation is similar to color tracking, which calculates linear velocity and angular velocity based on the center coordinates of the target and the depth information fed by the depth camera, and then releases them to the chassis.

Some of the code is as follows.

```
//After selecting an object, obtain the center coordinates for calculating angular velocity
if (bBeginKCF) {
    result = tracker.update(rgbimage);
    rectangle(rgbimage, result, Scalar(0, 255, 255), 1, 8);
    circle(rgbimage, Point(result.x + result.width / 2, result.y + result.height / 2), 3, Scalar(0, 0, 255), -1);
}
else rectangle(rgbimage, selectRect, Scalar(255, 0, 0), 2, 8, 0);

//Calculate the values of center_x and distance for calculating speed
int center_x = (int)(result.x + result.width / 2);
int num_depth_points = 5;
for (int i = 0; i < 5; i++) {
    if (dist_val[i] > 40 && dist_val[i] < 8000) distance += dist_val[i];
    else num_depth_points--;
}
if (num_depth_points == 0) distance = this->minDist;
else distance /= num_depth_points;

//Calculate linear velocity and angular velocity
linear_speed = -linear_PID->compute(this->minDist, distance);
rotation_speed = angular_PID->compute(320, center_x) / 1000.0;
```